

LE MANUEL CLAUDE

Niveau Un

De l'exécution de tâches à la construction de vraies choses.

Avec des schémas, des exemples concrets et des procédures pas-à-pas.

Prompting · Projects · Skills · Connectors · Données & RGPD · Source & Git · Code · Agents

La suite du Niveau Zéro — pour quand les bases vous semblent faciles.

Par Hans Tandt

Ardennes, Wallonie, Belgique

Version 2.1 — Août 2026

Comment utiliser ce livre

Ceci est le Niveau Un. Il suppose que vous avez fait les choses du Niveau Zéro : configurer Claude, lancer une tâche dans Cowork, retrouver vos fichiers, gérer une session. Sinon, lisez d'abord le Niveau Zéro — ce livre s'appuie dessus.

Le Niveau Un, c'est le passage de tâches ponctuelles à la construction de choses qui durent : mieux prompter, des skills réutilisables, des outils connectés, des données maîtrisées, du code source partagé, un pas dans Code, et votre premier petit agent.

Trois parcours — lisez ce dont vous avez besoin

Nouveau dans tout ceci : lisez dans l'ordre.

Vous avez les bases et voulez de la qualité : chapitres 1–5 (prompting, projects, skills, connectors, données & RGPD).

Vous avez besoin du travail d'équipe : allez aux chapitres 7–9 (source, Git, équipe) — ils se suffisent.

Ce livre est une photo à un instant — Claude change vite

Tout ici reflète Claude en août 2026 : les familles de modèles Opus, Sonnet, Haiku et Fable, l'app de bureau Cowork, et l'accueil unifié qui partage Chat, Cowork et vos Projects dans une seule barre latérale. Anthropic publie des mises à jour très vite — souvent en jours, pas en mois — donc un écran, un nom de menu ou un numéro de modèle a pu déjà bouger. Quand ce que vous voyez ne correspond pas à la page, fiez-vous à l'app et à claude.com. Les idées et les habitudes tiennent même quand les boutons changent de place.

Nouveautés de la version 2.1

Ajouts : « Record a skill » dans Cowork (chapitre 3) et Claude Design (chapitre 10).

Mises à jour : où vivent désormais les Projects dans le nouvel accueil unique, et un rappel que les modèles et les écrans changent vite.

Table des matières

Le Niveau Zéro en une page

Avant de construire, les cinq idées sur lesquelles tout repose :

1. **Choisissez le bon visage.** Chat pour réfléchir, Cowork pour produire, Code pour construire, Projects pour se souvenir, Connectors pour atteindre d'autres apps.
2. **Le contexte bat l'astuce.** Ce que Claude voit avant de répondre compte plus que la formulation.
3. **Laissez Claude demander d'abord, puis vérifiez.** « Pose-moi des questions d'abord » ; et ne croyez jamais une réponse assurée sans la vérifier.
4. **Un fichier n'est réel que dans votre dossier.** Tant qu'il n'y est pas, il vit dans le sandbox ou le scratchpad et peut disparaître.
5. **Gardez des sessions courtes.** Un sujet par session ; notes de session et mémoire pour ne jamais vous réexpliquer.

1. Le prompting qui marche vraiment

Le Niveau Zéro disait : le contexte bat l'astuce. Le Niveau Un le rend concret. Une bonne demande a quatre parties, et aucune n'est un mot magique.

Une bonne demande a quatre parties (pas un mot magique)



Donnez la destination et les standards — pas une longue liste d'étapes.

Partie	Ce que cela signifie	Exemple
Objectif	Le résultat voulu, en une phrase.	« Rédige une lettre d'offre d'une page. »
Contexte	Les faits, fichiers et contraintes nécessaires.	« Utilise le modèle joint et nos conditions habituelles. »
Exemples	Un ou deux échantillons de bonne (et mauvaise) sortie.	« Reprends le ton de cette ancienne lettre ; évite cette tournure. »
Niveau d'exigence	Comment vous jugerez ; ce que « fini » veut dire.	« Formel, moins de 250 mots, sans jargon. »

1.1 Le contexte en action — avant / après

Même tâche, deux prompts. **Avant** : « Écris un e-mail à un fournisseur sur la commande en retard. » Vous obtenez un e-mail générique. **Après** : « Écris à notre fournisseur Dupont, commande 4471, 9 jours de retard. Ferme mais poli, demande une nouvelle date et un avoir partiel, moins de 120 mots, signe en mon nom. » Vous obtenez quelque chose de presque prêt. La différence, ce n'est pas l'astuce — c'est le contexte.

1.2 Le fichier anti-style

Le moyen le plus rapide d'arrêter une sortie « qui sent l'IA » est d'écrire ce que vous ne voulez jamais. Un court fichier « anti-style » — mots à bannir, débuts détestés, tournures rejetées — fait plus que n'importe quelle consigne positive. Gardez-le dans ABOUT ME.

1.3 Montrer, puis corriger

Ne visez pas le coup parfait du premier essai. Demandez un brouillon, réagissez, laissez Claude ajuster. Trois tours de « bien, mais plus court et moins formel » battent trente minutes à fabriquer le prompt parfait.

1.4 Vérifier la réponse — quoi, et comment

Claude semble aussi sûr quand il a tort. Gardez Web Search activé, et demandez-lui de s'auto-vérifier : « *Vérifie ta propre réponse avant que je m'y fie ; contrôle les faits et dis-moi ce dont tu n'es pas sûr.* »

2. Projects en profondeur

Un Project est un dossier de conversations qui partagent instructions, fichiers et mémoire. Voici comment en faire un qui fait gagner du temps au lieu de devenir un fourre-tout. Depuis mi-2026, vos projects vivent dans la barre latérale de gauche de l'accueil unifié, partagés entre Chat et Cowork — épinglez ceux que vous utilisez pour les retrouver facilement.

2.1 Un project par livrable

L'erreur courante est un seul project géant pour « tout ». La solution : un project par livrable récurrent — un rapport hebdomadaire, un type de proposition, un outil précis. Chacun ne reçoit que ses fichiers et des instructions écrites pour ce seul travail.

2.2 Ce que contiennent les instructions

Les instructions ne sont pas de la documentation — c'est le minimum qui empêche Claude de dériver. Cinq à huit lignes :

- Ce qu'est ce livrable, et pour qui.
- Le ton et le niveau d'exigence.
- Ce qu'il faut toujours faire, et ne jamais faire.
- Où enregistrer le résultat, et dans quel format.

Les fichiers ne sont pas gratuits

Plus de fichiers ne veut pas dire de meilleures réponses — mais plus de bruit à filtrer. Envoyez des exemples de bon travail et vos modèles, pas toute votre archive. Si une réponse dérive, c'est souvent trop de fichiers, pas trop peu.

2.3 Procédure — créer un project utile en 5 minutes

1. Nommez-le d'après un livrable (« Rapport ops hebdo »).
2. N'ajoutez que ses modèles et 1–2 bons exemples passés.
3. Écrivez 5–8 lignes d'instructions (quoi / pour qui / ton / où enregistrer / quand demander).
4. Testez avec une vraie tâche ; faites-le s'auto-critiquer et resserrez les instructions.

3. Construire votre premier Skill

Un Skill est votre façon standard de faire un travail, écrite pour que Claude l'exécute pareil à chaque fois. Si vous avez déjà écrit une checklist, vous pouvez en construire un — aucun code requis.

3.1 Ce qu'est vraiment un skill

Un skill est un petit fichier texte nommé `SKILL.md`. Une courte description en haut, les instructions complètes en dessous. Seule la description reste en mémoire ; le reste se charge quand une tâche correspond. D'où la possibilité d'en avoir beaucoup sans rien ralentir.

3.2 Procédure — créer et tester un skill

1. **Faites la tâche une fois à la main** avec Claude, en notant les étapes et les erreurs.
2. **Récoltez-la** : « Transforme ce qu'on vient de faire en skill réutilisable. Écris les étapes clairement, ajoute une courte section de pièges, et garde la description sur une ligne. »
3. **Testez sur un exemple frais** légèrement différent, et corrigez ce que le skill a raté.
4. **Enregistrez-le** dans l'app, sous Settings, Capabilities. Un skill ne s'améliore que si on modifie son fichier.

Le plus rapide pour en créer un : filmez-vous en train de le faire

Depuis mi-2026, Cowork a une option « Record a skill » dans le menu + de l'app de bureau. Au lieu d'écrire les étapes, vous enregistrez votre écran et parlez de la tâche pendant que vous la faites une fois, et Claude transforme la démonstration en skill. C'est la voie la plus facile pour un travail plus simple à montrer qu'à décrire — surtout les tâches de navigateur sans connector. N'enregistrez ni mots de passe ni données privées. Sur Pro, Max et Team.

4. Connectors — laisser Claude atteindre vos autres outils

Un connector est un pont sécurisé entre Claude et une autre app — courrier, agenda, un drive, un suivi de projet. Avec un connector actif, Claude peut lire et agir sur des données hors de votre dossier.

4.1 Ne reliez que ce dont la tâche a besoin

L'instinct est de tout relier. Résistez. Chaque connector ajoute du bruit et consomme du contexte. Un briefing du matin a besoin du courrier et de l'agenda — pas de toute votre boîte à outils. Désactivez ceux qu'un flux n'utilise pas.

4.2 Les trois sortes de connector

Les connectors reposent sur un standard commun, **MCP** (Model Context Protocol). Vous les ajoutez depuis les réglages Connectors (le « + » dans une conversation, ou Settings, Connectors). Il en existe trois sortes, et la différence compte pour la confiance :

- **Les intégrations intégrées d'Anthropic** — de première partie (Gmail, Drive, GitHub, Slack...).
- **Les connectors de l'annuaire** — tiers, mais examinés et validés par Anthropic avant d'être listés.
- **Les connectors MCP personnalisés que vous ajoutez vous-même** — personne ne les a examinés ; même prudence qu'avec toute nouvelle app.

4.3 Un premier usage sûr

Commencez en lecture seule. Demandez de résumer le courrier non lu du jour, ou de lister les réunions de la semaine, avant de laisser Claude envoyer ou modifier. Bâissez la confiance sur des tâches sans conséquence, puis élargissez.

5. Données, sécurité et RGPD avant de connecter

Dès que Claude touche de vraies données d'entreprise — via un connector, un document collé ou un dossier — la protection des données devient votre responsabilité. Ce chapitre est de la sensibilisation pratique, pas un conseil juridique ; en cas de doute, demandez à votre délégué à la protection des données (DPO).

5.1 La check-list données avant de connecter

1. Quelles données Claude verra-t-il, et certaines identifient-elles une personne ?
2. Ai-je une raison (une base légale) de les traiter ainsi ?
3. Est-ce le minimum nécessaire, ou est-ce que j'en partage trop ?
4. Pourrais-je anonymiser ou caviarder et obtenir quand même le résultat ?

5.2 Le RGPD en langage simple

Le RGPD de l'UE encadre les données personnelles — tout ce qui identifie une personne vivante (noms, e-mails, dossiers de personnel ou de clients, un dossier d'accident).

Quelques principes couvrent l'essentiel :

- **Ayez une raison.** Ne traitez des données personnelles que pour un but clair et légitime.
- **Le minimum.** N'introduisez pas plus de détails personnels que la tâche n'en exige.
- **Restez dans le but.** Ne réutilisez pas les données pour quelque chose d'inattendu.
- **Protégez et supprimez.** Gardez-les en sécurité, et pas plus longtemps que nécessaire.

Où vont les données ? (et puis-je les garder en local ?)

Chat, Cowork et Code envoient tous votre contenu aux serveurs d'Anthropic. Claude ne tourne pas sur votre ordinateur, et aucun réglage ne garde vos données en local. Vous POUVEZ désactiver l'usage de vos conversations pour entraîner les modèles (Settings, Privacy), ce qui raccourcit aussi la conservation. Les comptes Team, Entreprise et API n'entraînent pas sur vos données par défaut. Pour une IA vraiment locale et hors ligne, il faudrait un autre modèle auto-hébergé — pas Claude.

5.3 Tracer et garder la main

Pour tout ce qui est sensible, gardez une trace légère de ce que vous avez fourni et pourquoi, préférez la règle du feu tricolore du Niveau Zéro (vert / orange / rouge), et assurez-vous qu'un humain vérifie avant tout envoi ou action. Si un connector peut agir (envoyer du courrier, modifier des dossiers), commencez en lecture seule.

Pas un conseil juridique

Ce chapitre est une sensibilisation générale pour éviter les erreurs évidentes. Les obligations RGPD dépendent de votre rôle et de votre organisation — confirmez les détails avec votre DPO avant de traiter des données personnelles à grande échelle.

6. Mener plusieurs projets en parallèle

Vous n'êtes pas limité à une chose à la fois. Vous pouvez écrire un document dans Cowork pendant que Claude corrige une app dans Code, et garder plusieurs tâches Cowork en mouvement. Les outils tournent indépendamment — la limite, c'est vous, et votre quota.

6.1 Ce qui peut tourner en même temps

- **Cowork et Code ensemble.** Deux fenêtres, deux travaux, sans interférence.
- **Plusieurs sessions.** Différentes conversations ou projects, chacune avec son contexte, côte à côte.
- **Tâches d'arrière-plan et programmées.** Du travail qui avance sans surveillance, et même des tâches Dispatch depuis le téléphone.

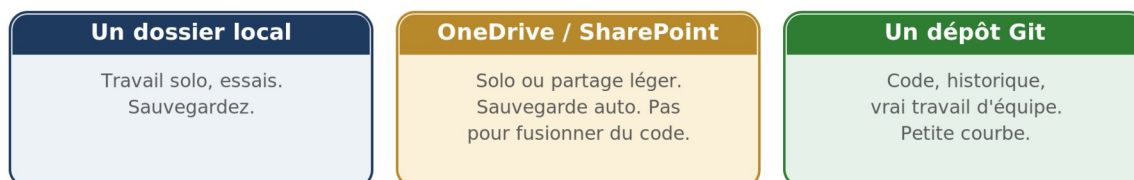
6.2 Les deux vrais coûts

1. **Votre attention.** Les machines ne se mélangent pas, mais vous, si. Sauter d'un document à un bug toutes les minutes est fatigant. Regroupez par contexte : une heure d'écriture, puis une heure sur l'app.
2. **Votre quota.** Deux gros travaux puisent dans les mêmes limites (5 h et hebdomadaire). Les mener ensemble dépense votre quota plus vite.

7. Où vit le code source de votre projet

Dès qu'un projet compte — ou que plusieurs personnes y touchent — l'endroit où vivent les fichiers cesse d'être un détail.

Où doit vivre le code source de votre projet



Seul sur des docs → local + OneDrive. Code, ou une équipe → Git.

7.1 Une règle simple

- **Seul sur des documents ?** Un dossier local sauvegardé par OneDrive suffit.
- **Seul sur du code ?** Utilisez Git quand même — un historique complet et un « annuler » pour tout le projet.
- **À plusieurs sur les mêmes fichiers ?** Git pour le code ; une bibliothèque SharePoint ou Drive pour les documents.

7.2 Gardez une structure nette

Quel que soit votre choix, pointez Cowork ou Code vers un dossier propre à la forme évidente : fichiers source, un endroit pour les sorties, un fichier de notes ou README, et rien d'étranger. Un dossier net rend Claude plus rapide et vos collègues plus sereins.

8. Une introduction en douceur à Git

Git donne à un projet une mémoire et un filet de sécurité — la chose la plus utile à apprendre en passant du solo au travail partagé, et bien moins effrayante qu'il n'y paraît.

8.1 Ce que fait vraiment Git

Git prend des instantanés de votre projet quand vous le demandez. Chaque instantané (un « commit ») est un point étiqueté où revenir : vous pouvez donc tout changer sans crainte, car si ça casse, vous revenez au dernier bon instantané.

Mot	En clair
Repository (dépôt)	Votre dossier de projet, plus tout son historique d'instantanés.
Commit	Un instantané enregistré, avec un court message.
Push / Pull	Envoyer vos commits / récupérer ceux des autres.
Branch (branche)	Une voie de côté sûre pour essayer sans toucher à la version principale.
Clone	Faire votre propre copie locale d'un dépôt partagé.

8.2 La boucle quotidienne

La boucle Git quotidienne — quatre étapes, répétées



Dites à Claude dans Code : « commit avec un message sensé et push ».

GitHub (ou GitLab, ou Azure DevOps en environnement Microsoft) héberge la copie partagée. Vous pouvez demander à Claude dans Code d'exécuter les étapes, mais comprendre ce qu'elles font aide.

Deux règles qui évitent l'essentiel des soucis Git

- 1) Pull avant de commencer, et committez souvent par petits bouts — faciles à comprendre et à annuler.
- 2) Jamais de secrets (clés d'API, mots de passe) dans un dépôt : une fois poussés, considérez-les publics. Gardez-les dans un fichier local que Git ignore.

8.3 Atelier — votre premier dépôt en 10 minutes

1. Dans Code, ouvrez un dossier de projet et dites « configure ce dossier comme dépôt Git ».
2. Faites un petit changement, puis « commit avec un message clair ».
3. Créez un compte GitHub gratuit ; dites « crée un dépôt privé et pousse-le ».
4. Changez encore, commit, push — et voyez l'historique se construire.

9. Travailler en équipe sur le même projet

Dès que deux personnes touchent aux mêmes fichiers, il faut un moyen de combiner leur travail sans s'écraser.

9.1 Documents ou code

- **Documents partagés** (Word, Excel, slides) : une bibliothèque SharePoint ou Drive suit les versions et montre qui a changé quoi. Évitez d'éditer le même fichier à la même seconde.
- **Code partagé** (scripts, apps, texte) : utilisez Git, conçu pour fusionner les changements de plusieurs personnes sans risque.

9.2 La boucle Git en équipe

1. **Pull d'abord** pour bâtir sur le travail des autres.
2. **Travaillez sur une branche** nommée d'après la tâche, pas sur la version principale.
3. **Ouvrez une pull request** — un collègue y jette un œil, puis elle rejoint la version principale.

Cette revue n'est pas de la bureaucratie — c'est une deuxième paire d'yeux, et là que vit la compréhension partagée. Quand deux personnes changent les mêmes lignes, Git demande quelle version garder (un « merge conflict ») ; cela paraît alarmant mais c'est routinier, et vous pouvez demander à Claude de l'expliquer et de le résoudre.

10. Un premier pas en douceur dans Code

Claude Code écrit et exécute de vrais logiciels. Vous n'avez pas à lire ni écrire le code — vous décrivez ce que vous voulez et Claude le fait.

10.1 Décrire, ne pas taper

« Ajoute un bouton qui exporte le tableau vers Excel » est une instruction complète. Claude propose le changement, vous l'approuvez, il l'exécute. Vous restez le décideur ; Claude tape.

10.2 CLAUDE.md — le cerveau du projet

Tout project Code devrait avoir un fichier `CLAUDE.md` à sa racine. Lu au début de chaque session, il contient la structure du projet, comment le lancer, vos choix, vos préférences. Après une bonne première session, dites « écris un `CLAUDE.md` résumant tout ce que tu as appris » et tenez-le à jour.

10.3 Les permissions — trois niveaux, pas deux

- **Demander à chaque fois** (par défaut) : sûr, lent.
- **Auto-accepter les éditions** mais confirmer les commandes risquées : un bon compromis.
- **Ignorer toutes les permissions** (mode « vibe » rapide) : seulement dans un dossier dédié, sauvegardé par Git, remplaçable — jamais votre dossier personnel.

10.4 Planifier d'abord, construire ensuite

Pour tout ce qui n'est pas trivial, demandez à Claude d'enquêter et de proposer un court plan avant d'écrire une ligne — « montre-moi deux options et attends mon choix ». Approuver un plan évite les longs détours et vous garde aux commandes.

10.5 Vibe coding — et ses limites

Ce que vous faites ici porte un nom : le vibe coding — construire un logiciel en décrivant ce que vous voulez et en pilotant au feeling, plutôt qu'en lisant et écrivant le code. Il brille pour les outils personnels, prototypes et scripts jetables. Pour ce dont d'autres dépendent, les règles se resserrent : gardez-le dans Git, exécutez et testez vraiment, et ne livrez jamais ce que vous ne pouvez pas vérifier. Vibe-codez librement sur les enjeux faibles ; ralentissez et vérifiez sur tout ce qui compte.

10.6 Claude Design — des interfaces sans code

Cousin de Code, lancé en 2026 en aperçu de recherche sur claude.ai/design, Claude Design construit l'apparence des choses — maquettes, landing pages, sites simples et présentations — à partir d'une description en clair. Contrairement à un générateur d'images, il produit la vraie chose : une mise en page fonctionnelle avec des boutons cliquables, pas une image de bouton. Utilisez-le quand ce que vous voulez est une page ou un design plutôt qu'un logiciel. Sur Pro, Max, Team et Enterprise ; étant un aperçu de recherche, attendez-vous à ce qu'il évolue.

11. Votre premier petit agent

Un agent n'est que Claude travaillant vers un but en plusieurs étapes, souvent sur un horaire, et faisant son rapport. Pas besoin d'un grand système — une tâche utile et répétée est déjà un agent.

11.1 Ce qui fait un agent

Trois ingrédients : un travail clair, un déclencheur (une heure ou un événement), et un moyen de vous dire le résultat. Le reste n'est que détail.

11.2 Un cas complet — un briefing matinal

1. **Définissez le travail** : « Chaque jour ouvré à 7 h, liste mes réunions du jour et le courrier non lu marqué important, et écris un résumé de 5 lignes. »
2. **Faites-en une tâche enregistrée** pour qu'elle s'exécute pareil, avec les connecteurs courrier + agenda en lecture seule.
3. **Programmez-la** à 07 h les jours ouvrés (votre machine, ou une machine toujours allumée, doit être éveillée).
4. **Envoyez-la là où vous la verrez** — un e-mail à vous-même, ou un message sur votre téléphone.

Commencez par un essai à blanc

Lancez-la à la main quelques fois et lisez la sortie d'un œil critique avant de programmer. Un agent qui répète une petite erreur chaque matin est pire que pas d'agent.

11.3 Faites-le grandir lentement

Ajoutez une capacité à la fois, et gardez un court fichier de notes sur ce qu'il fait et comment le réparer. Ce n'est pas parce que Claude peut ajouter une fonction en quelques secondes qu'elle devrait exister.

12. Outils et apps à installer

Vous pouvez aller loin avec la seule app de bureau. Mais quelques outils gratuits et standards rendent la construction plus fluide. Installez-les au fur et à mesure.

Outil	À quoi il sert	Quand il vous le faut
Un éditeur (VS Code, ou Notepad++)	Lire et modifier légèrement fichiers texte et code.	Dès que vous ouvrez vous-même les fichiers d'un projet.
Git + un compte GitHub	Historique de versions et travail d'équipe (ch. 8–9).	Dès qu'un projet compte ou est partagé.
Python et Node.js	Les moteurs de la plupart des petits outils.	Quand Code construit quelque chose à exécuter en local.
PuTTY / WinSCP	Se connecter à un serveur ou Pi et y copier des fichiers.	Quand vous faites tourner des tâches sur une machine allumée.
Un bot de messagerie (Telegram)	Laisser un agent vous envoyer alertes et rapports.	Quand vous voulez des résultats sur votre téléphone.

Laissez Claude les installer

Dites « guide-moi pour installer Git sur Windows, étape par étape, et vérifie chaque étape avec moi ». Pareil pour Python, Node, ou la connexion à un Pi.

13. Rester maintenable

La différence entre un week-end amusant et un système fiable dans six mois, ce sont quelques habitudes discrètes.

- **Sauvegardez à deux endroits.** Le source dans Git, les dossiers importants synchronisés ou zippés. Une copie n'est pas une sauvegarde.
- **Committez et notez au fil de l'eau.** De petits commits et un README court : le vous futur comprend vite un projet.
- **Mémoire et notes de session à grande échelle.** Un MEMORY.md et des notes par projet orientent chaque nouvelle session.
- **Élaguez skills et connectors** que vous n'utilisez plus — les morts sont encombrement et risque.
- **Sachez vous arrêter.** Avant d'ajouter une fonction, demandez si elle mérite sa place. La soustraction est un savoir-faire.

Le test d'une phrase

Pour tout projet, vous devriez pouvoir dire en une phrase ce qu'il fait et pourquoi il existe. Sinon, il a grandi sans direction — élaguez avant d'étendre.

Glossaire des nouveaux termes

Terme	En mots simples
Repository (dépôt)	Un dossier de projet plus tout son historique d'instantanés, géré par Git.
Commit	Un instantané enregistré, avec un court message décrivant le changement.
Push / Pull	Envoyer vos commits vers la copie en ligne / récupérer ceux des autres.
Branch (branche)	Une voie de côté sûre pour essayer sans toucher à la version principale.
Merge conflict	Quand deux personnes ont changé les mêmes lignes ; Git demande quoi garder. Routinier.
Pull request	Une demande de fusion de votre branche, généralement revue d'abord.
GitHub / GitLab	Des sites qui hébergent la copie partagée d'un dépôt Git.
CLAUDE.md	Un fichier à la racine d'un project Code qui dit à Claude comment il fonctionne.
MCP	Le standard commun que les connecteurs utilisent pour relier Claude à d'autres apps.
RGPD	Les règles de l'UE sur les données personnelles — tout ce qui identifie une personne vivante.
Agent	Claude travaillant vers un but en plusieurs étapes, souvent programmé, faisant son rapport.

La suite

Si vous avez traversé le Niveau Zéro et ce livre Niveau Un, vous savez bien configurer Claude, produire de vrais livrables, gérer les données de façon responsable, garder votre source sûr et partagé, faire un pas dans Code et Claude Design, et faire tourner un petit agent. C'est l'essentiel de ce dont la plupart des gens ont besoin.

L'API — l'étape au-delà de ce livre

Vous avez peut-être entendu parler de l'**API Claude** (qui fait partie du Claude Platform). L'app, Cowork et Code vous laissent tous utiliser Claude à travers une fenêtre construite par d'autres. L'API permet à un programme à vous de parler directement à Claude — facturé au token — pour construire vos propres apps et agents. C'est plus technique, d'où sa place au-delà de ce livre ; mais c'est plus proche qu'il n'y paraît — dès qu'un petit outil que vous faites dans Code appelle Claude tout seul, il utilise déjà l'API.

Et gardez près de vous les deux habitudes du Niveau Zéro : **laissez Claude demander d'abord, et ne croyez jamais une réponse assurée sans la vérifier**. Elles comptent davantage, pas moins, à mesure que ce que vous construisez grandit.

Par Hans Tandt